```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
import pandas as pd

DATA_URL = (
    "https://raw.githubusercontent.com/"
    "IBM/employee-attrition-aif360/master/data/emp_attrition.csv"
)

try:
    df = pd.read_csv(DATA_URL)
    print("Dataset loaded successfully!")
except Exception as e:
    print("Could not load the dataset:", e)
    print("Upload your CSV manually using the next alternative cell.")

if "df" in globals():
    print("Dataset shape:", df.shape)
    display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (1470, 35)
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNu
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	

5 rows × 35 columns

```
from google.colab import files
import io
import pandas as pd

uploaded = files.upload()

csv_files = [name for name in uploaded if name.lower().endswith(".csv")]

if not csv_files:
    raise ValueError("Please upload a CSV file.")

df = pd.read_csv(io.BytesIO(uploaded[csv_files[0]]))

print("Dataset loaded:", csv_files[0])
print("Dataset shape:", df.shape)
display(df.head())
```

Choose Files HR_Attritio…_Dataset.csv

HR_Attrition_Indian_Dataset.csv(text/csv) - 617977 bytes, last modified: 8/24/2026 - 100% done

```
Saving HR_Attrition_Indian_Dataset.csv to HR_Attrition_Indian_Dataset.csv
Dataset loaded: HR_Attrition_Indian_Dataset.csv
Dataset shape: (5000, 22)
```

	EmployeeID	FullName	Age	Gender	Department	Designation	EducationQualification	MonthlySalary	DateOfJoining	YearsWith
0	EMP20230001	Sahil Tailor	48	Other	HR	Accounts Officer	B.Tech	111813	2020-05-26	
1	EMP20230002	Parinaaz Choudhary	26	Female	IT	Accounts Officer	B.Tech	87425	2020-10-02	
2	EMP20230003	Anahi Babu	40	Male	IT	Software Engineer	Diploma	86461	2021-05-01	
3	EMP20230004	Dhruv Dass	54	Other	HR	Accounts Officer	Diploma	23720	2023-04-01	
4	EMP20230005	Seher Bal	51	Other	Admin	Sales Associate	B.Tech	39041	2018-12-19	

5 rows × 22 columns

```
print("Number of employees:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types and non-null counts:")
df.info()

print("\nFirst five records:")
display(df.head())

print("\nStatistical summary:")
display(df.describe(include="all").T)
```

```
Number of employees: 5000
Number of columns: 22

Column names:
['EmployeeID', 'FullName', 'Age', 'Gender', 'Department', 'Designation', 'EducationQualification', 'MonthlySalary', 'DateOfJoi

Data types and non-null counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 22 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   EmployeeID              5000 non-null   object
 1   FullName                5000 non-null   object
 2   Age                     5000 non-null   int64
 3   Gender                  5000 non-null   object
```

```
print("Missing values per column:")
display(df.isnull().sum().sort_values(ascending=False))

print("Duplicate rows:", df.duplicated().sum())

print("\nUnique values per column:")
display(df.nunique().sort_values().to_frame("Unique Values"))

print("\nDataset shape before cleaning:", df.shape)
```

```
 15  WorkLifeBalance         5000 non-null   int64
 16  MaritalStatus           5000 non-null   object
 17  PreviousCompanies       5000 non-null   int64
 18  DistanceFromHome        5000 non-null   float64
 19  LeavesTaken             5000 non-null   int64
 20  City                    5000 non-null   object
 21  LeftCompany             5000 non-null   object
dtypes: float64(1), int64(10), object(11)
memory usage: 859.5+ KB

First five records:
```

	EmployeeID	FullName	Age	Gender	Department	Designation	EducationQualification	MonthlySalary	DateOfJoining	YearsWi
0	EMP20230001	Sahil Tailor	48	Other	HR	Accounts Officer	B.Tech	111813	2020-05-26	
1	EMP20230002	Parinaaz Choudhary	26	Female	IT	Accounts Officer	B.Tech	87425	2020-10-02	
2	EMP20230003	Anahi Babu	40	Male	IT	Software Engineer	Diploma	86461	2021-05-01	
3	EMP20230004	Dhruv Dass	54	Other	HR	Accounts Officer	Diploma	23720	2023-04-01	
4	EMP20230005	Seher Bal	51	Other	Admin	Sales Associate	B.Tech	39041	2018-12-19	

5 rows × 22 columns

Statistical summary:

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
EmployeeID	5000	5000	EMP20235000	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
FullName	5000	4851	Vihaan Swaminathan	2	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Age	5000.0	NaN	NaN	NaN	40.71	11.361079	22.0	31.0	41.0	51.0	60.0
Gender	5000	3	Female	1695	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Department	5000	5	Accounts	1031	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Designation	5000	5	HR Executive	1049	NaN	NaN	NaN	NaN	NaN	NaN	NaN
EducationQualification	5000	5	M.Com	1029	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MonthlySalary	5000.0	NaN	NaN	NaN	68200.8962	30290.742577	15031.0	42086.0	68358.5	94889.5	119878.0
DateOfJoining	5000	2593	2015-07-16	7	NaN	NaN	NaN	NaN	NaN	NaN	NaN
YearsWithCompany	5000.0	NaN	NaN	NaN	5.5786	2.615414	1.0	3.0	6.0	8.0	10.0
YearsInRole	5000.0	NaN	NaN	NaN	2.8204	2.443305	0.0	1.0	2.0	4.0	10.0
AppraisalRating	5000.0	NaN	NaN	NaN	2.9882	1.40216	1.0	2.0	3.0	4.0	5.0
TrainingHours	5000.0	NaN	NaN	NaN	30.0778	17.55775	0.0	15.0	30.0	45.0	60.0

DoesOvertime	5000	2	No	2539	NaN	NaN	NaN	NaN	NaN	NaN	NaN
JobSatisfaction	5000.0	NaN	NaN	NaN	2.9908	1.407662	1.0	2.0	3.0	4.0	5.0
WorkLifeBalance	5000.0	NaN	NaN	NaN	2.5022	1.11868	1.0	2.0	2.5	4.0	4.0
MaritalStatus	5000	3	Married	1730	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PreviousCompanies	5000.0	NaN	NaN	NaN	2.5088	1.718753	0.0	1.0	3.0	4.0	5.0
DistanceFromHome	5000.0	NaN	NaN	NaN	25.47184	14.19387	1.0	13.175	25.5	37.7	50.0
LeavesTaken	5000.0	NaN	NaN	NaN	14.9178	8.972912	0.0	7.0	15.0	23.0	30.0
City	5000	7	Mumbai	745	NaN	NaN	NaN	NaN	NaN	NaN	NaN
LeftCompany	5000	2	No	3145	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
Missing values per column:
```

	0
EmployeeID	0
FullName	0

```
if "Department" in df.columns:
    print("Employees by department:")
    display(df["Department"].value_counts().to_frame("Employee Count"))

    df["Department"].value_counts().plot(
        kind="bar",
        figsize=(8, 4),
        title="Employee Distribution by Department"
    )
    plt.xlabel("Department")
    plt.ylabel("Number of Employees")
    plt.xticks(rotation=30)
    plt.tight_layout()
    plt.show()

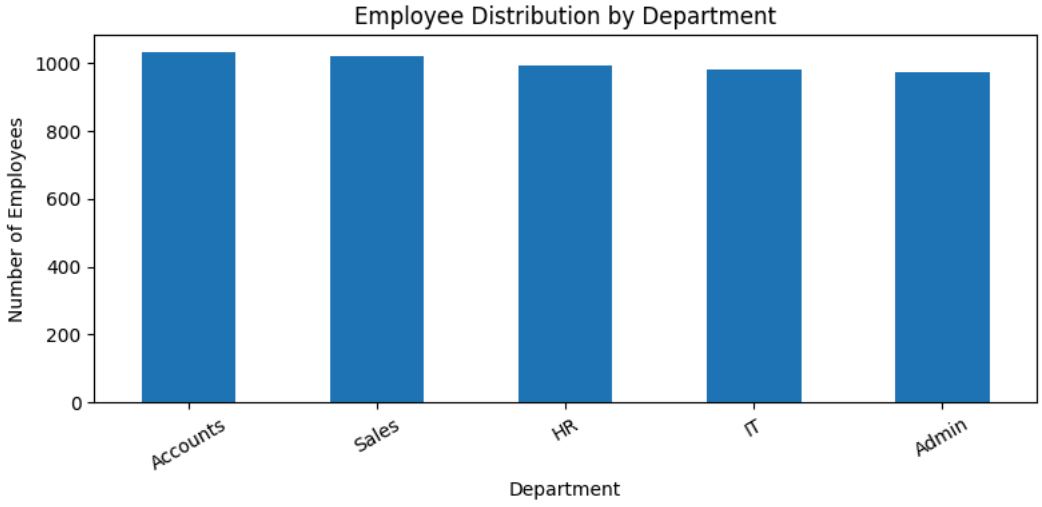
if "Attrition" in df.columns:
    print("Attrition distribution:")
    display(df["Attrition"].value_counts().to_frame("Employee Count"))

    sns.countplot(data=df, x="Attrition")
    plt.title("Employee Attrition Distribution")
    plt.tight_layout()
    plt.show()
```

```
Employees by department:
```

Department	Employee Count
Accounts	1031
Sales	1019
HR	992
IT	983
Admin	975

```
dtype: int64
Duplicate rows: 0
```




```
import pandas as pd
import numpy as np

# Keep the original dataset unchanged
df_clean = df.copy()

# Remove duplicate records
df_clean = df_clean.drop_duplicates().copy()

# Remove columns that contain only one unique value
constant_cols = [
```

```
    col for col in df_clean.columns
    if df_clean[col].nunique(dropna=False) <= 1
]

df_clean.drop(columns=constant_cols, inplace=True)

print("Constant columns removed:", constant_cols)
print("Dataset shape after cleaning:", df_clean.shape)
print("Duplicate rows remaining:", df_clean.duplicated().sum())
print("Missing values remaining:", df_clean.isnull().sum().sum())

display(df_clean.head())
```

```
Constant columns removed: []
Dataset shape after cleaning: (5000, 22)
Duplicate rows remaining: 0
Missing values remaining: 0
```

	EmployeeID	FullName	Age	Gender	Department	Designation	EducationQualification	MonthlySalary	DateOfJoining	YearsWith
0	EMP20230001	Sahil Tailor	48	Other	HR	Accounts Officer	B.Tech	111813	2020-05-26	
1	EMP20230002	Parinaaz Choudhary	26	Female	IT	Accounts Officer	B.Tech	87425	2020-10-02	
2	EMP20230003	Anahi Babu	40	Male	IT	Software Engineer	Diploma	86461	2021-05-01	
3	EMP20230004	Dhruv Dass	54	Other	HR	Accounts Officer	Diploma	23720	2023-04-01	
4	EMP20230005	Seher Bal	51	Other	Admin	Sales Associate	B.Tech	39041	2018-12-19	

5 rows × 22 columns

```
# Detect likely identifier columns
id_candidates = [
    col for col in df_clean.columns
    if col.lower().replace(" ", "").replace("_", "")
    in ["employeeid", "employeenumber", "id"]
]

# Preserve attrition separately if available
target_col = next(
    (col for col in df_clean.columns
     if col.lower().replace(" ", "").replace("_", "") == "attrition"),
    None
)

employee_ids = (
    df_clean[id_candidates[0]].copy()
    if id_candidates else pd.Series(df_clean.index, index=df_clean.index)
)

attrition_labels = (
    df_clean[target_col].copy()
    if target_col else None
)

# Exclude IDs and the attrition label from clustering features
exclude_cols = id_candidates.copy()
if target_col:
    exclude_cols.append(target_col)

feature_df = df_clean.drop(columns=exclude_cols, errors="ignore").copy()

print("Identifier columns excluded:", id_candidates)
print("Attrition label preserved separately:", target_col)
print("Features available for clustering:", feature_df.shape[1])
print(feature_df.columns.tolist())
```

```
Identifier columns excluded: ['EmployeeID']
Attrition label preserved separately: None
Features available for clustering: 21
['FullName', 'Age', 'Gender', 'Department', 'Designation', 'EducationQualification', 'MonthlySalary', 'DateOfJoining', 'YearsWit
```

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Identify numerical and categorical columns
numeric_cols = feature_df.select_dtypes(
    include=["number", "bool"]
).columns.tolist()

categorical_cols = feature_df.select_dtypes(
    include=["object", "category", "string"]
).columns.tolist()

# Numerical preprocessing
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical preprocessing
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_cols),
    ("categorical", categorical_pipeline, categorical_cols)
])

X_processed = preprocessor.fit_transform(feature_df)

# Get readable transformed feature names
feature_names = preprocessor.get_feature_names_out()

X_processed = pd.DataFrame(
    X_processed,
    columns=feature_names,
    index=feature_df.index
)

print("Preprocessing completed successfully!")
print("Processed dataset shape:", X_processed.shape)
print("Missing values:", X_processed.isnull().sum().sum())

display(X_processed.head())
```

```
Preprocessing completed successfully!
Processed dataset shape: (5000, 7487)
Missing values: 0
```

	numeric__Age	numeric__MonthlySalary	numeric__YearsWithCompany	numeric__YearsInRole	numeric__AppraisalRating	numeric__Tra
0	0.641729	1.439927	-0.221249	-1.154454	0.008416	
1	-1.294901	0.634716	-0.221249	-0.335808	0.008416	
2	-0.062500	0.602888	-0.603636	0.482837	-0.704840	
3	1.169900	-1.468612	-1.368410	-0.335808	-1.418097	
4	0.905814	-0.962763	0.543525	-0.745131	-1.418097	

5 rows × 7487 columns

```
import os

os.makedirs("employee_workforce_segmentation/data/processed",
            exist_ok=True)

processed_path = (
    "employee_workforce_segmentation/data/processed/"
    "processed_employee_features.csv"
```

```
)

X_processed.to_csv(processed_path, index=False)

print("Processed data saved to:", processed_path)
print("Number of employee records:", len(X_processed))
print("Number of processed features:", X_processed.shape[1])
```

```
Processed data saved to: employee_workforce_segmentation/data/processed/processed_employee_features.csv
Number of employee records: 5000
Number of processed features: 7487
```

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# Test different cluster counts
max_k = min(8, len(X_processed) - 1)
k_values = list(range(2, max_k + 1))

inertia_values = []
silhouette_values = []

for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_processed)

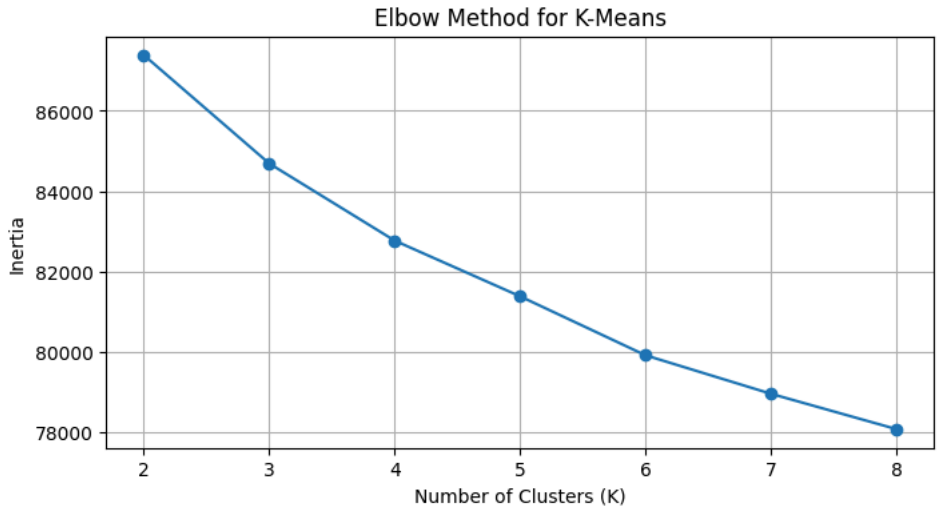
    inertia_values.append(model.inertia_)
    silhouette_values.append(
        silhouette_score(X_processed, labels)
    )

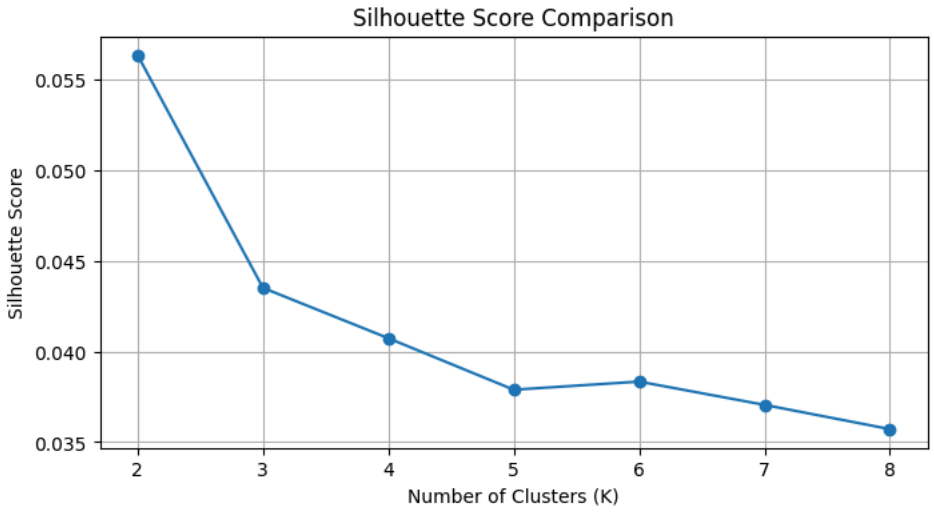
# Plot Elbow Method
plt.figure(figsize=(8, 4))
plt.plot(k_values, inertia_values, marker="o")
plt.title("Elbow Method for K-Means")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(k_values)
plt.grid(True)
plt.show()

# Plot Silhouette Scores
plt.figure(figsize=(8, 4))
plt.plot(k_values, silhouette_values, marker="o")
plt.title("Silhouette Score Comparison")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.xticks(k_values)
plt.grid(True)
plt.show()

best_k = k_values[silhouette_values.index(max(silhouette_values))]

print("Best K based on silhouette score:", best_k)
print("Highest silhouette score:", round(max(silhouette_values), 4))
```






```
Best K based on silhouette score: 2
Highest silhouette score: 0.0563
```

```
from sklearn.cluster import KMeans

kmeans_model = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

kmeans_labels = kmeans_model.fit_predict(X_processed)

# Keep the cluster labels with each employee
employee_segments = df_clean.copy()
employee_segments["KMeans_Cluster"] = kmeans_labels

print("K-Means training completed!")
print("\nEmployees in each cluster:")
print(employee_segments["KMeans_Cluster"].value_counts().sort_index())

display(employee_segments.head())
```

```
K-Means training completed!

Employees in each cluster:
KMeans_Cluster
0    2219
1    2781
Name: count, dtype: int64
```

	EmployeeID	FullName	Age	Gender	Department	Designation	EducationQualification	MonthlySalary	DateOfJoining	YearsWith
0	EMP20230001	Sahil Tailor	48	Other	HR	Accounts Officer	B.Tech	111813	2020-05-26	
1	EMP20230002	Parinaaz Choudhary	26	Female	IT	Accounts Officer	B.Tech	87425	2020-10-02	
2	EMP20230003	Anahi Babu	40	Male	IT	Software Engineer	Diploma	86461	2021-05-01	
3	EMP20230004	Dhruv Dass	54	Other	HR	Accounts Officer	Diploma	23720	2023-04-01	
4	EMP20230005	Seher Bal	51	Other	Admin	Sales Associate	B.Tech	39041	2018-12-19	

5 rows × 23 columns

```
from sklearn.cluster import AgglomerativeClustering

hierarchical_model = AgglomerativeClustering(
    n_clusters=best_k,
    linkage="ward"
)

hierarchical_labels = hierarchical_model.fit_predict(X_processed)

employee_segments["Hierarchical_Cluster"] = hierarchical_labels

print("Hierarchical clustering completed!")
print("\nEmployees in each hierarchical cluster:")
print(
    employee_segments["Hierarchical_Cluster"]
    .value_counts()
    .sort_index()
)

display(employee_segments.head())
```

```
Hierarchical clustering completed!

Employees in each hierarchical cluster:
Hierarchical_Cluster
0    3164
1    1836
Name: count, dtype: int64
```

	EmployeeID	FullName	Age	Gender	Department	Designation	EducationQualification	MonthlySalary	DateOfJoining	YearsWith
0	EMP20230001	Sahil Tailor	48	Other	HR	Accounts Officer	B.Tech	111813	2020-05-26	
1	EMP20230002	Parinaaz Choudhary	26	Female	IT	Accounts Officer	B.Tech	87425	2020-10-02	
2	EMP20230003	Anahi Babu	40	Male	IT	Software Engineer	Diploma	86461	2021-05-01	
3	EMP20230004	Dhruv Dass	54	Other	HR	Accounts Officer	Diploma	23720	2023-04-01	
4	EMP20230005	Seher Bal	51	Other	Admin	Sales Associate	B.Tech	39041	2018-12-19	

5 rows × 24 columns

```
from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)
```

```
import pandas as pd

comparison = pd.DataFrame({
    "Model": ["K-Means", "Hierarchical"],
    "Silhouette Score": [
        silhouette_score(X_processed, kmeans_labels),
        silhouette_score(X_processed, hierarchical_labels)
    ],
    "Davies-Bouldin Index": [
        davies_bouldin_score(X_processed, kmeans_labels),
        davies_bouldin_score(X_processed, hierarchical_labels)
    ],
    "Calinski-Harabasz Score": [
        calinski_harabasz_score(X_processed, kmeans_labels),
        calinski_harabasz_score(X_processed, hierarchical_labels)
    ]
})

display(comparison.round(4))
```

	Model	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	K-Means	0.0563	3.9916	308.2215
1	Hierarchical	0.0385	4.8010	200.0117

```
from sklearn.decomposition import PCA
import seaborn as sns
import matplotlib.pyplot as plt

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_processed)

pca_df = pd.DataFrame({
    "PC1": X_pca[:, 0],
    "PC2": X_pca[:, 1],
    "KMeans_Cluster": kmeans_labels,
    "Hierarchical_Cluster": hierarchical_labels
}, index=X_processed.index)

print("Variance explained by PC1 and PC2:")
print(pca.explained_variance_ratio_.round(4))

print(
    "Total variance explained:",
    round(pca.explained_variance_ratio_.sum() * 100, 2),
    "%"
)

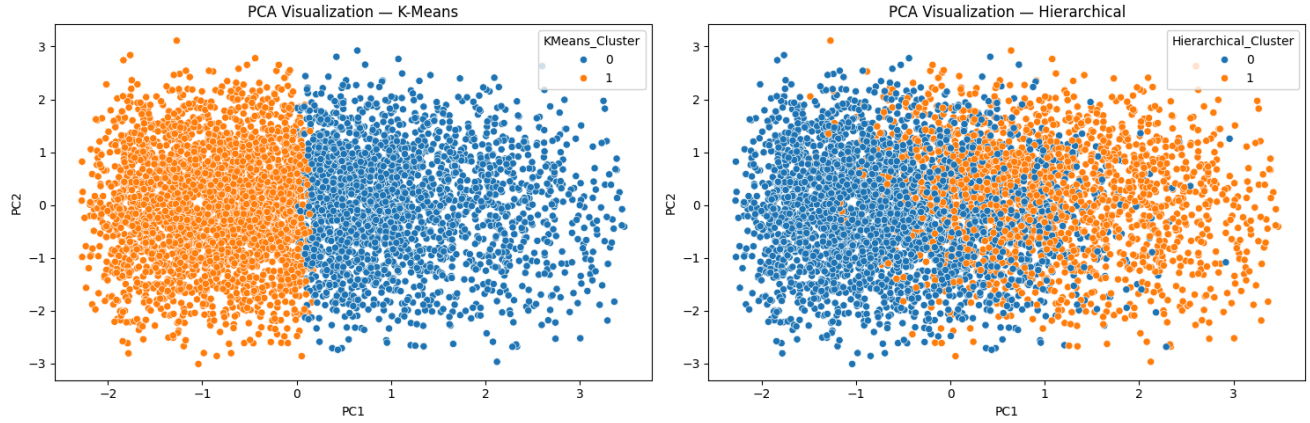
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="KMeans_Cluster",
    palette="tab10",
    s=35,
    ax=axes[0]
)
axes[0].set_title("PCA Visualization – K-Means")

sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="Hierarchical_Cluster",
    palette="tab10",
    s=35,
    ax=axes[1]
)
axes[1].set_title("PCA Visualization – Hierarchical")

plt.tight_layout()
plt.show()
```

```
Variance explained by PC1 and PC2:
[0.0838 0.0598]
Total variance explained: 14.36 %
```




```
# Cluster sizes
print("Cluster sizes:")
display(
    employee_segments["KMeans_Cluster"]
    .value_counts()
    .sort_index()
    .rename("Employee Count")
    .to_frame()
)

# Select useful columns that exist in the dataset
profile_candidates = [
    "Age",
    "MonthlyIncome",
    "MonthlyRate",
    "JobSatisfaction",
    "EnvironmentSatisfaction",
    "RelationshipSatisfaction",
    "WorkLifeBalance",
    "PerformanceRating",
    "YearsAtCompany",
    "YearsInCurrentRole",
    "JobLevel",
    "DistanceFromHome"
]

numeric_profile_cols = [
    col for col in profile_candidates
    if col in employee_segments.columns
]

if numeric_profile_cols:
    print("Average employee characteristics by cluster:")
    display(
        employee_segments.groupby("KMeans_Cluster")[
            numeric_profile_cols
        ].mean().round(2)
    )

if "Department" in employee_segments.columns:
    print("Department distribution within each cluster:")
    display(
        pd.crosstab(
            employee_segments["KMeans_Cluster"],
            employee_segments["Department"],
            normalize="index"
        ).round(3) * 100
```

```
    )

if "Attrition" in employee_segments.columns:
    print("Historical attrition rate by cluster:")
    attrition_rate = (
        employee_segments.assign(
            Attrition_Flag=(
                employee_segments["Attrition"]
                .astype(str).str.strip().str.lower()
                .map({"yes": 1, "no": 0})
            )
        )
        .groupby("KMeans_Cluster")["Attrition_Flag"]
        .mean()
        .mul(100)
        .round(2)
    )

    display(attrition_rate.rename(
        "Historical Attrition (%)"
    ).to_frame())
```

```
Cluster sizes:
```

KMeans_Cluster	Employee Count
0	2219
1	2781

```
Average employee characteristics by cluster:
```

KMeans_Cluster	Age	JobSatisfaction	WorkLifeBalance	DistanceFromHome
0	41.44	3.06	2.47	25.21
1	40.13	2.94	2.53	25.68

```
Department distribution within each cluster:
```

KMeans_Cluster	Department	Accounts	Admin	HR	IT	Sales
0		19.7	19.3	20.5	20.5	20.1
1		21.4	19.7	19.3	19.0	20.6

```
import os

os.makedirs("employee_workforce_segmentation/results", exist_ok=True)

# Save employee labels and original data
employee_segments.to_csv(
    "employee_workforce_segmentation/results/"
    "employee_segmentation_results.csv",
    index=False
)

# Save model comparison metrics
comparison.to_csv(
    "employee_workforce_segmentation/results/"
    "clustering_model_comparison.csv",
    index=False
)

# Save PCA coordinates
pca_df.to_csv(
    "employee_workforce_segmentation/results/"
    "pca_employee_visualization.csv",
    index=False
)

print("Clustering results saved successfully!")
```

```
Clustering results saved successfully!
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Department distribution across K-Means clusters
if "Department" in employee_segments.columns:
    department_table = pd.crosstab(
        employee_segments["Department"],
        employee_segments["KMeans_Cluster"]
    )

    print("Department-wise employee count by cluster:")
    display(department_table)

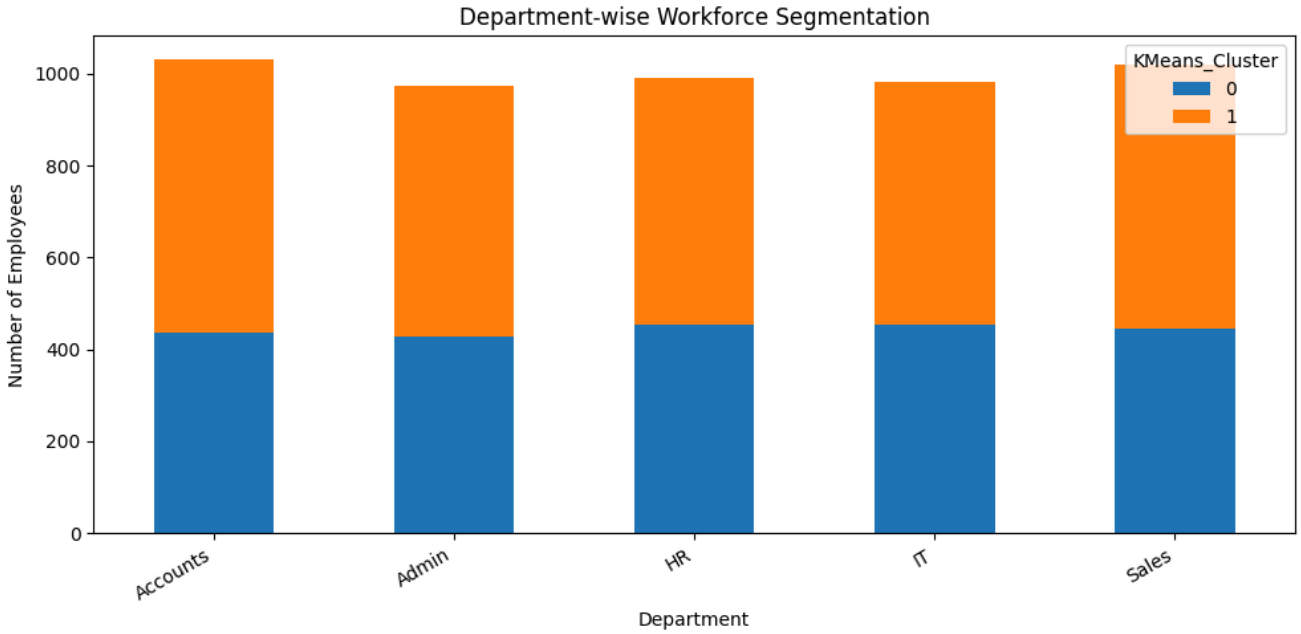
    department_table.plot(
        kind="bar",
        stacked=True,
        figsize=(10, 5)
    )
    plt.title("Department-wise Workforce Segmentation")
    plt.xlabel("Department")
    plt.ylabel("Number of Employees")
    plt.xticks(rotation=30, ha="right")
    plt.tight_layout()
    plt.show()

    # Percentage distribution within each department
    department_pct = pd.crosstab(
        employee_segments["Department"],
        employee_segments["KMeans_Cluster"],
        normalize="index"
    ).mul(100).round(2)

    print("Cluster percentages within each department:")
    display(department_pct)
else:
    print("Department column not available in this dataset.")
```

Department-wise employee count by cluster:

Department	KMeans_Cluster 0	1
Accounts	437	594
Admin	428	547
HR	454	538
IT	454	529
Sales	446	573



Cluster percentages within each department:

Department	KMeans_Cluster 0	1
Accounts	42.39	57.61
Admin	43.90	56.10
HR	45.77	54.23
IT	46.19	53.81
Sales	43.77	56.23

Next steps: Generate code with department_table | New interactive sheet | Generate code with department_pct | New interactive sheet

```
# Analyze available employee performance and satisfaction fields
analysis_cols = [
    "PerformanceRating",
    "JobSatisfaction",
    "EnvironmentSatisfaction",
    "RelationshipSatisfaction",
    "WorkLifeBalance",
    "JobLevel",
    "YearsAtCompany",
    "MonthlyIncome"
]

available_cols = [
    col for col in analysis_cols
    if col in employee_segments.columns
]

if available_cols:
    cluster_profile = (
        employee_segments
        .groupby("KMeans_Cluster")[available_cols]
        .mean()
```

```
        .round(2)
    )

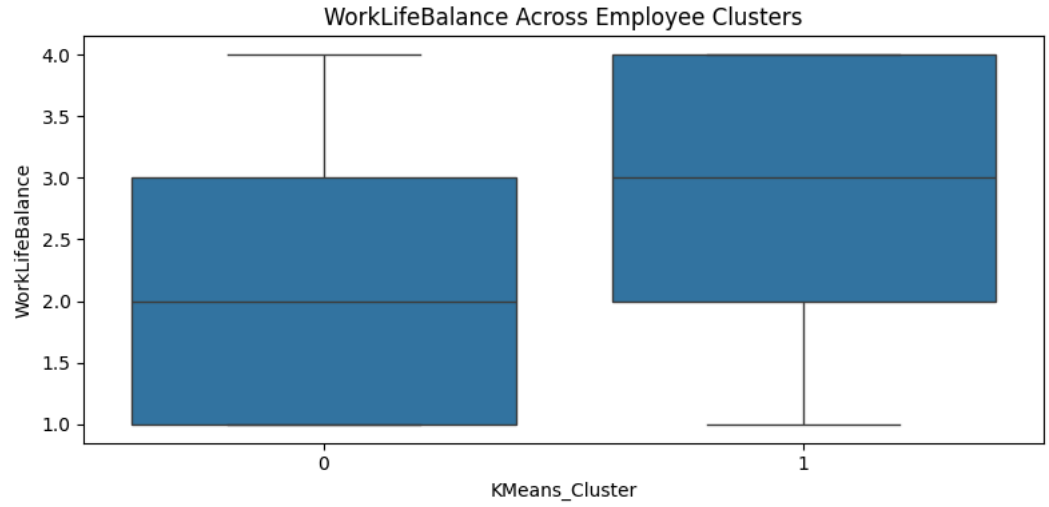
    print("Employee performance and satisfaction by cluster:")
    display(cluster_profile)

    # Visualize fields that exist in the dataset
    for col in [
        "PerformanceRating",
        "JobSatisfaction",
        "WorkLifeBalance"
    ]:
        if col in employee_segments.columns:
            plt.figure(figsize=(8, 4))
            sns.boxplot(
                data=employee_segments,
                x="KMeans_Cluster",
                y=col
            )
            plt.title(f"{col} Across Employee Clusters")
            plt.tight_layout()
            plt.show()
else:
    print("No matching performance or satisfaction columns found.")
```

```
Employee performance and satisfaction by cluster:
```

KMeans_Cluster	JobSatisfaction	WorkLifeBalance
0	3.06	2.47
1	2.94	2.53





Next steps: Generate code with `cluster_profile` New interactive sheet

```
if "Attrition" in employee_segments.columns:

    attrition_text = (
        employee_segments["Attrition"]
        .astype("string")
        .str.strip()
        .str.lower()
    )

    # Handle common yes/no and true/false representations
    attrition_flag = attrition_text.map({
        "yes": 1,
        "no": 0,
        "true": 1,
        "false": 0,
        "1": 1,
        "0": 0
    })

    retention_df = employee_segments.copy()
    retention_df["Attrition_Flag"] = attrition_flag

    retention_summary = (
        retention_df
        .groupby("KMeans_Cluster")
        .agg(
            Employee_Count=("Attrition_Flag", "size"),
            Known_Attrition_Records=("Attrition_Flag", "count"),
            Historical_Attrition_Rate=("Attrition_Flag", "mean")
        )
    )

    retention_summary["Historical_Attrition_Percent"] = (
        retention_summary["Historical_Attrition_Rate"] * 100
    ).round(2)

    overall_rate = attrition_flag.mean() * 100

    print(f"Overall historical attrition rate: {overall_rate:.2f}%")
    print("Historical attrition by cluster:")
    display(retention_summary.round(3))

    plt.figure(figsize=(8, 4))
    retention_summary["Historical_Attrition_Percent"].plot(
        kind="bar"
    )
    plt.axhline(
        overall_rate,
        linestyle="--",
        label="Overall dataset rate"
    )
    plt.title("Historical Attrition Rate by Employee Cluster")
    plt.xlabel("K-Means Cluster")
    plt.ylabel("Historical Attrition (%)")
    plt.legend()
    plt.tight_layout()
    plt.show()

else:
    print(
        "No Attrition column found. "
        "Historical retention analysis requires an attrition label."
    )
```

```
No Attrition column found. Historical retention analysis requires an attrition label.
```

```
import os

output_dir = "employee_workforce_segmentation/results"
os.makedirs(output_dir, exist_ok=True)

# Export department segmentation
if "Department" in employee_segments.columns:
    department_table.to_csv(
        f"{output_dir}/department_segmentation.csv"
    )
```

```
# Export performance and satisfaction profiles
if available_cols:
    cluster_profile.to_csv(
        f"{output_dir}/performance_satisfaction_analysis.csv"
    )

# Export retention analysis
if "Attrition" in employee_segments.columns:
    retention_summary.to_csv(
        f"{output_dir}/retention_analysis.csv"
    )

print("Workforce analytics reports exported successfully.")

for filename in os.listdir(output_dir):
    print("-", filename)
```

```
Workforce analytics reports exported successfully.
- pca_employee_visualization.csv
- employee_segmentation_results.csv
- department_segmentation.csv
- clustering_model_comparison.csv
- performance_satisfaction_analysis.csv
```

```
!pip -q install streamlit plotly
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.4/10.4 MB 101.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.4/11.4 MB 119.4 MB/s eta 0:00:00
```

```
%%writefile app.py
import os
import pandas as pd
import streamlit as st
import plotly.express as px

st.set_page_config(
    page_title="Employee Workforce Segmentation",
    page_icon="📊",
    layout="wide"
)

RESULTS_DIR = "employee_workforce_segmentation/results"

st.title("Employee Workforce Segmentation")
st.caption(
    "Workforce analytics | Clustering | PCA | Department insights | Retention"
)

@st.cache_data
def load_csv(path):
    return pd.read_csv(path)

results_path = os.path.join(
    RESULTS_DIR, "employee_segmentation_results.csv"
)
pca_path = os.path.join(
    RESULTS_DIR, "pca_employee_visualization.csv"
)
comparison_path = os.path.join(
    RESULTS_DIR, "clustering_model_comparison.csv"
)

if not os.path.exists(results_path):
    st.error(
        "Employee segmentation results were not found. "
        "Place employee_segmentation_results.csv inside "
        "employee_workforce_segmentation/results/."
    )
    st.stop()

df = load_csv(results_path)

if df.empty:
    st.error("The employee segmentation file is empty.")
    st.stop()
```

```
# Sidebar filters
st.sidebar.header("Dashboard Filters")

if "Department" in df.columns:
    departments = sorted(
        df["Department"].dropna().astype(str).unique().tolist()
    )
    selected_departments = st.sidebar.multiselect(
        "Select department",
        departments,
        default=departments
    )
    filtered = df[
        df["Department"].astype(str).isin(selected_departments)
    ].copy()
else:
    filtered = df.copy()
    st.sidebar.info("Department field is not available.")

cluster_options = []
if "KMeans_Cluster" in filtered.columns:
    cluster_options = sorted(
        filtered["KMeans_Cluster"].dropna().unique().tolist()
    )

if cluster_options:
    selected_clusters = st.sidebar.multiselect(
        "Select K-Means clusters",
        cluster_options,
        default=cluster_options
    )
    filtered = filtered[
        filtered["KMeans_Cluster"].isin(selected_clusters)
    ].copy()

if filtered.empty:
    st.warning("No employees match the selected filters.")
    st.stop()

# KPI metrics
st.header("Workforce Overview")

c1, c2, c3, c4 = st.columns(4)
c1.metric("Employees", f"{len(filtered):,}")

if "Department" in filtered.columns:
    c2.metric("Departments", filtered["Department"].nunique())
else:
    c2.metric("Departments", "N/A")

if "KMeans_Cluster" in filtered.columns:
    c3.metric("K-Means Clusters", filtered["KMeans_Cluster"].nunique())
else:
    c3.metric("K-Means Clusters", "N/A")

if "MonthlyIncome" in filtered.columns:
    c4.metric(
        "Average Monthly Income",
        f"{filtered['MonthlyIncome'].mean():,.0f}"
    )
else:
    c4.metric("Average Monthly Income", "N/A")

# Cluster distribution
st.header("Employee Segmentation")

left, right = st.columns(2)

with left:
    if "KMeans_Cluster" in filtered.columns:
        cluster_counts = (
            filtered["KMeans_Cluster"]
            .value_counts()
            .sort_index()
            .rename_axis("Cluster")
            .reset_index(name="Employees")
```

```
            )
            fig = px.bar(
                cluster_counts,
                x="Cluster",
                y="Employees",
                title="K-Means Cluster Distribution",
                text="Employees"
            )
            st.plotly_chart(fig, use_container_width=True)

    with right:
        if "Hierarchical_Cluster" in filtered.columns:
            hierarchy_counts = (
                filtered["Hierarchical_Cluster"]
                .value_counts()
                .sort_index()
                .rename_axis("Cluster")
                .reset_index(name="Employees")
            )
            fig = px.bar(
                hierarchy_counts,
                x="Cluster",
                y="Employees",
                title="Hierarchical Cluster Distribution",
                text="Employees"
            )
            st.plotly_chart(fig, use_container_width=True)

    # Department segmentation
    if "Department" in filtered.columns and "KMeans_Cluster" in filtered.columns:
        st.header("Department-Level Workforce Segments")

        department_counts = pd.crosstab(
            filtered["Department"],
            filtered["KMeans_Cluster"]
        ).reset_index()

        department_long = department_counts.melt(
            id_vars="Department",
            var_name="Cluster",
            value_name="Employees"
        )

        fig = px.bar(
            department_long,
            x="Department",
            y="Employees",
            color="Cluster",
            barmode="stack",
            title="Department Distribution by K-Means Cluster"
        )
        st.plotly_chart(fig, use_container_width=True)

    # PCA visualization
    st.header("PCA Cluster Visualization")

    if os.path.exists(pca_path):
        pca_df = load_csv(pca_path)

        if {"PC1", "PC2"}.issubset(pca_df.columns):
            label_col = "KMeans_Cluster"
            if label_col not in pca_df.columns:
                label_col = "Hierarchical_Cluster"

            if label_col in pca_df.columns:
                # Align PCA rows to the employee table by original row order.
                # This works when both exports retain the same row order.
                if len(pca_df) == len(df):
                    selected_positions = df.index[
                        df.index.isin(filtered.index)
                    ]
                    pca_filtered = pca_df.iloc[
                        selected_positions.tolist()
                    ].copy()
                else:
                    pca_filtered = pca_df.copy()
```

```
            fig = px.scatter(
                pca_filtered,
                x="PC1",
                y="PC2",
                color=label_col,
                title="Employees Projected onto Two Principal Components",
                opacity=0.75
            )
            st.plotly_chart(fig, use_container_width=True)
            st.caption(
                "PCA is a two-dimensional projection of the processed "
                "features. Overlapping points do not necessarily indicate "
                "identical employees."
            )
        else:
            st.info("Cluster labels were not found in the PCA file.")
    else:
        st.info("The PCA file must contain PC1 and PC2 columns.")
else:
    st.info("PCA results file is not available.")

# Performance and satisfaction
profile_cols = [
    "PerformanceRating",
    "JobSatisfaction",
    "EnvironmentSatisfaction",
    "WorkLifeBalance",
    "YearsAtCompany",
    "MonthlyIncome"
]
available_profile_cols = [
    col for col in profile_cols if col in filtered.columns
]

if available_profile_cols and "KMeans_Cluster" in filtered.columns:
    st.header("Performance and Employee Characteristics")
    profile = (
        filtered.groupby("KMeans_Cluster")[available_profile_cols]
        .mean()
        .round(2)
    )
    st.dataframe(profile, use_container_width=True)

# Historical attrition
if "Attrition" in filtered.columns:
    st.header("Historical Attrition Analysis")

    attrition = (
        filtered["Attrition"]
        .astype(str)
        .str.strip()
        .str.lower()
        .map({
            "yes": 1, "no": 0,
            "true": 1, "false": 0,
            "1": 1, "0": 0
        })
    )

    known = filtered.assign(Attrition_Flag=attrition).dropna(
        subset=["Attrition_Flag"]
    )

    if not known.empty:
        overall_rate = known["Attrition_Flag"].mean() * 100
        st.metric("Historical Attrition Rate", f"{overall_rate:.2f}%")

        if "KMeans_Cluster" in known.columns:
            rates = (
                known.groupby("KMeans_Cluster")["Attrition_Flag"]
                .mean()
                .mul(100)
                .reset_index(name="Historical Attrition (%)")
            )
            fig = px.bar(
                rates,
                x="KMeans_Cluster",
```

```
                y="Historical Attrition (%)",
                title="Historical Attrition by Cluster"
            )
            st.plotly_chart(fig, use_container_width=True)

        st.caption(
            "These are historical group-level statistics, not predictions "
            "of individual employees' future behavior."
        )
    else:
        st.info("No recognizable attrition labels were found.")

# Model comparison
if os.path.exists(comparison_path):
    st.header("Clustering Model Comparison")
    comparison = load_csv(comparison_path)
    st.dataframe(comparison, use_container_width=True)

# Employee data and download
st.header("Employee Segmentation Results")
st.dataframe(filtered, use_container_width=True)

st.download_button(
    "Download Filtered Employee Data (CSV)",
    data=filtered.to_csv(index=False).encode("utf-8"),
    file_name="employee_segmentation_results.csv",
    mime="text/csv"
)

st.success("Dashboard data loaded successfully.")
```

```
Writing app.py
```

```
import os

required_files = [
    "app.py",
    "employee_workforce_segmentation/results/employee_segmentation_results.csv",
    "employee_workforce_segmentation/results/pca_employee_visualization.csv",
    "employee_workforce_segmentation/results/clustering_model_comparison.csv"
]

for path in required_files:
    print(
        "OK" if os.path.exists(path) else "MISSING",
        path
    )
```

```
OK app.py
OK employee_workforce_segmentation/results/employee_segmentation_results.csv
OK employee_workforce_segmentation/results/pca_employee_visualization.csv
OK employee_workforce_segmentation/results/clustering_model_comparison.csv
```

```
import os

print("Current directory:", os.getcwd())

for root, dirs, files in os.walk("/content"):
    dirs[:] = [d for d in dirs if d not in [".config", ".cache"]]
    for file in files:
        if file.endswith(".csv"):
            print(os.path.join(root, file))
```

```
Current directory: /content
/content/HR_Attrition_Indian_Dataset.csv
/content/employee_workforce_segmentation/data/processed/processed_employee_features.csv
/content/employee_workforce_segmentation/results/pca_employee_visualization.csv
/content/employee_workforce_segmentation/results/employee_segmentation_results.csv
/content/employee_workforce_segmentation/results/department_segmentation.csv
/content/employee_workforce_segmentation/results/clustering_model_comparison.csv
/content/employee_workforce_segmentation/results/performance_satisfaction_analysis.csv
/content/sample_data/mnist_train_small.csv
/content/sample_data/california_housing_test.csv
/content/sample_data/california_housing_train.csv
/content/sample_data/mnist_test.csv
```

```
import os
import shutil

target = "/content/employee_workforce_segmentation/results"
os.makedirs(target, exist_ok=True)

required = [
    "employee_segmentation_results.csv",
    "pca_employee_visualization.csv",
    "clustering_model_comparison.csv"
]

for filename in required:
    matches = []

    for root, dirs, files in os.walk("/content"):
        dirs[:] = [
            d for d in dirs
            if d not in [".config", ".cache"]
        ]

        if filename in files:
            source = os.path.join(root, filename)
            if os.path.abspath(source) != os.path.abspath(
                os.path.join(target, filename)
            ):
                matches.append(source)

    destination = os.path.join(target, filename)

    if os.path.exists(destination):
        print("OK:", filename)
    elif matches:
        shutil.copy2(matches[0], destination)
        print("Restored:", filename, "from", matches[0])
    else:
        print("MISSING:", filename)

print("\nFinal results folder:")
print(os.listdir(target))
```

```
OK: employee_segmentation_results.csv
OK: pca_employee_visualization.csv
OK: clustering_model_comparison.csv

Final results folder:
['pca_employee_visualization.csv', 'employee_segmentation_results.csv', 'department_segmentation.csv', 'clustering_model_compari
```

```
import os

print("Results available:", os.listdir(
    "/content/employee_workforce_segmentation/results"
))
print("Dashboard exists:", os.path.exists("/content/app.py"))
```

```
Results available: ['pca_employee_visualization.csv', 'employee_segmentation_results.csv', 'department_segmentation.csv', 'clust
Dashboard exists: True
```

```
import os
import pandas as pd
import streamlit as st
import plotly.express as px

st.set_page_config(
    page_title="Employee Workforce Segmentation",
    page_icon="📊",
    layout="wide"
)

RESULTS_DIR = "employee_workforce_segmentation/results"

st.title("Employee Workforce Segmentation")
st.caption(
    "Workforce analytics | Clustering | PCA | Department insights | Retention"
)
```

```
@st.cache_data
def load_csv(path):
    return pd.read_csv(path)

results_path = os.path.join(
    RESULTS_DIR, "employee_segmentation_results.csv"
)
pca_path = os.path.join(
    RESULTS_DIR, "pca_employee_visualization.csv"
)
comparison_path = os.path.join(
    RESULTS_DIR, "clustering_model_comparison.csv"
)

if not os.path.exists(results_path):
    st.error(
        "Employee segmentation results were not found. "
        "Place employee_segmentation_results.csv inside "
        "employee_workforce_segmentation/results/."
    )
    st.stop()

df = load_csv(results_path)

if df.empty:
    st.error("The employee segmentation file is empty.")
    st.stop()

# Sidebar filters
st.sidebar.header("Dashboard Filters")

if "Department" in df.columns:
    departments = sorted(
        df["Department"].dropna().astype(str).unique().tolist()
    )
    selected_departments = st.sidebar.multiselect(
        "Select department",
        departments,
        default=departments
    )
    filtered = df[
        df["Department"].astype(str).isin(selected_departments)
    ].copy()
else:
    filtered = df.copy()
    st.sidebar.info("Department field is not available.")

cluster_options = []
if "KMeans_Cluster" in filtered.columns:
    cluster_options = sorted(
        filtered["KMeans_Cluster"].dropna().unique().tolist()
    )

if cluster_options:
    selected_clusters = st.sidebar.multiselect(
        "Select K-Means clusters",
        cluster_options,
        default=cluster_options
    )
    filtered = filtered[
        filtered["KMeans_Cluster"].isin(selected_clusters)
    ].copy()

if filtered.empty:
    st.warning("No employees match the selected filters.")
    st.stop()

# KPI metrics
st.header("Workforce Overview")

c1, c2, c3, c4 = st.columns(4)
c1.metric("Employees", f"{len(filtered):,}")

if "Department" in filtered.columns:
    c2.metric("Departments", filtered["Department"].nunique())
else:
    c2.metric("Departments", "N/A")
```

```
if "KMeans_Cluster" in filtered.columns:
    c3.metric("K-Means Clusters", filtered["KMeans_Cluster"].nunique())
else:
    c3.metric("K-Means Clusters", "N/A")

if "MonthlyIncome" in filtered.columns:
    c4.metric(
        "Average Monthly Income",
        f"{filtered['MonthlyIncome'].mean():,.0f}"
    )
else:
    c4.metric("Average Monthly Income", "N/A")

# Cluster distribution
st.header("Employee Segmentation")

left, right = st.columns(2)

with left:
    if "KMeans_Cluster" in filtered.columns:
        cluster_counts = (
            filtered["KMeans_Cluster"]
            .value_counts()
            .sort_index()
            .rename_axis("Cluster")
            .reset_index(name="Employees")
        )
        fig = px.bar(
            cluster_counts,
            x="Cluster",
            y="Employees",
            title="K-Means Cluster Distribution",
            text="Employees"
        )
        st.plotly_chart(fig, use_container_width=True)

with right:
    if "Hierarchical_Cluster" in filtered.columns:
        hierarchy_counts = (
            filtered["Hierarchical_Cluster"]
            .value_counts()
            .sort_index()
            .rename_axis("Cluster")
            .reset_index(name="Employees")
        )
        fig = px.bar(
            hierarchy_counts,
            x="Cluster",
            y="Employees",
            title="Hierarchical Cluster Distribution",
            text="Employees"
        )
        st.plotly_chart(fig, use_container_width=True)

# Department segmentation
if "Department" in filtered.columns and "KMeans_Cluster" in filtered.columns:
    st.header("Department-Level Workforce Segments")

    department_counts = pd.crosstab(
        filtered["Department"],
        filtered["KMeans_Cluster"]
    ).reset_index()

    department_long = department_counts.melt(
        id_vars="Department",
        var_name="Cluster",
        value_name="Employees"
    )

    fig = px.bar(
        department_long,
        x="Department",
        y="Employees",
        color="Cluster",
        barmode="stack",
        title="Department Distribution by K-Means Cluster"
    )
```

```
    st.plotly_chart(fig, use_container_width=True)

# PCA visualization
st.header("PCA Cluster Visualization")

if os.path.exists(pca_path):
    pca_df = load_csv(pca_path)

    if {"PC1", "PC2"}.issubset(pca_df.columns):
        label_col = "KMeans_Cluster"
        if label_col not in pca_df.columns:
            label_col = "Hierarchical_Cluster"

        if label_col in pca_df.columns:
            # Align PCA rows to the employee table by original row order.
            # This works when both exports retain the same row order.
            if len(pca_df) == len(df):
                selected_positions = df.index[
                    df.index.isin(filtered.index)
                ]
                pca_filtered = pca_df.iloc[
                    selected_positions.tolist()
                ].copy()
            else:
                pca_filtered = pca_df.copy()

            fig = px.scatter(
                pca_filtered,
                x="PC1",
                y="PC2",
                color=label_col,
                title="Employees Projected onto Two Principal Components",
                opacity=0.75
            )
            st.plotly_chart(fig, use_container_width=True)
            st.caption(
                "PCA is a two-dimensional projection of the processed "
                "features. Overlapping points do not necessarily indicate "
                "identical employees."
            )
        else:
            st.info("Cluster labels were not found in the PCA file.")
    else:
        st.info("The PCA file must contain PC1 and PC2 columns.")
else:
    st.info("PCA results file is not available.")

# Performance and satisfaction
profile_cols = [
    "PerformanceRating",
    "JobSatisfaction",
    "EnvironmentSatisfaction",
    "WorkLifeBalance",
    "YearsAtCompany",
    "MonthlyIncome"
]
available_profile_cols = [
    col for col in profile_cols if col in filtered.columns
]

if available_profile_cols and "KMeans_Cluster" in filtered.columns:
    st.header("Performance and Employee Characteristics")
    profile = (
        filtered.groupby("KMeans_Cluster")[available_profile_cols]
        .mean()
        .round(2)
    )
    st.dataframe(profile, use_container_width=True)

# Historical attrition
if "Attrition" in filtered.columns:
    st.header("Historical Attrition Analysis")

    attrition = (
        filtered["Attrition"]
        .astype(str)
        .str.strip()
```

```
        .str.lower()
        .map({
            "yes": 1, "no": 0,
            "true": 1, "false": 0,
            "1": 1, "0": 0
        })
    )

    known = filtered.assign(Attrition_Flag=attrition).dropna(
        subset=["Attrition_Flag"]
    )

    if not known.empty:
        overall_rate = known["Attrition_Flag"].mean() * 100
        st.metric("Historical Attrition Rate", f"{overall_rate:.2f}%")

        if "KMeans_Cluster" in known.columns:
            rates = (
                known.groupby("KMeans_Cluster")["Attrition_Flag"]
                .mean()
                .mul(100)
                .reset_index(name="Historical Attrition (%)")
            )
            fig = px.bar(
                rates,
                x="KMeans_Cluster",
                y="Historical Attrition (%)",
                title="Historical Attrition by Cluster"
            )
            st.plotly_chart(fig, use_container_width=True)

        st.caption(
            "These are historical group-level statistics, not predictions "
            "of individual employees' future behavior."
        )
    else:
        st.info("No recognizable attrition labels were found.")

# Model comparison
if os.path.exists(comparison_path):
    st.header("Clustering Model Comparison")
    comparison = load_csv(comparison_path)
    st.dataframe(comparison, use_container_width=True)

# Employee data and download
st.header("Employee Segmentation Results")
st.dataframe(filtered, use_container_width=True)

st.download_button(
    "Download Filtered Employee Data (CSV)",
    data=filtered.to_csv(index=False).encode("utf-8"),
    file_name="employee_segmentation_results.csv",
    mime="text/csv"
)

st.success("Dashboard data loaded successfully.")
```

```
2026-10-20 13:03:49.351 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.353 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.355 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.357 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.357 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.358 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.363 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-10-20 13:03:49.367 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.368 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.369 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.370 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.371 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.373 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.373 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.374 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-10-20 13:03:49.397 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.398 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.399 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.400 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.451 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.452 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.452 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.453 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.454 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.455 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.455 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.457 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-10-20 13:03:49.457 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
DeltaGenerator()
```

```
import os

os.makedirs(
    "employee_workforce_segmentation/results",
    exist_ok=True
)

print("Folder created successfully!")
```

```
Folder created successfully!
```

```
import os

# Create the results folder
folder = "employee_workforce_segmentation/results"
os.makedirs(folder, exist_ok=True)

# Save all three results
employee_segments.to_csv(
    f"{folder}/employee_segmentation_results.csv",
    index=False
)

comparison.to_csv(
    f"{folder}/clustering_model_comparison.csv",
    index=False
)

pca_df.to_csv(
    f"{folder}/pca_employee_visualization.csv",
    index=False
)

print("All three files saved successfully!")

print(os.listdir(folder))
```

```
All three files saved successfully!
['pca_employee_visualization.csv', 'employee_segmentation_results.csv', 'department_segmentation.csv', 'clustering_model_compari
```

```
import os

folder = "employee_workforce_segmentation/results"

for filename in os.listdir(folder):
    print(filename, "— Saved successfully")
```

```
pca_employee_visualization.csv — Saved successfully
employee_segmentation_results.csv — Saved successfully
department_segmentation.csv — Saved successfully
clustering_model_comparison.csv — Saved successfully
performance_satisfaction_analysis.csv — Saved successfully
```

```
import os

folder = "/content/employee_workforce_segmentation/results"

required_files = [
    "employee_segmentation_results.csv",
    "clustering_model_comparison.csv",
    "pca_employee_visualization.csv"
]

for filename in required_files:
    path = os.path.join(folder, filename)
    print(
        "✅ FOUND:" if os.path.exists(path) else "❌ MISSING:",
        filename
    )

print("\nDashboard file exists:", os.path.exists("/content/app.py"))
```

```
✅ FOUND: employee_segmentation_results.csv
✅ FOUND: clustering_model_comparison.csv
✅ FOUND: pca_employee_visualization.csv

Dashboard file exists: True
```

```
import os

print("Dashboard file exists:", os.path.exists("/content/app.py"))
```

```
Dashboard file exists: True
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

# Use the results you already created
data = employee_segments.copy()

# Dashboard heading
print("=" * 55)
print("       EMPLOYEE WORKFORCE SEGMENTATION")
print("=" * 55)

# KPI cards (text-based)
print("\nWORKFORCE OVERVIEW")
print("Total Employees:", len(data))
print("Departments:", data["Department"].nunique()
      if "Department" in data.columns else "Not available")
print("K-Means Clusters:", data["KMeans_Cluster"].nunique())
print("Hierarchical Clusters:",
      data["Hierarchical_Cluster"].nunique())

# 1. Cluster distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.countplot(
    data=data, x="KMeans_Cluster",
    ax=axes[0]
)
axes[0].set_title("K-Means Employee Segments")

sns.countplot(
    data=data, x="Hierarchical_Cluster",
```

```
    ax=axes[1]
)
axes[1].set_title("Hierarchical Employee Segments")

plt.tight_layout()
plt.show()

# 2. PCA visualization
plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue="KMeans_Cluster",
    palette="tab10"
)
plt.title("Employee Segmentation using PCA")
plt.tight_layout()
plt.show()

# 3. Department analysis
if "Department" in data.columns:
    dept_table = pd.crosstab(
        data["Department"],
        data["KMeans_Cluster"]
    )
    print("\nDEPARTMENT-WISE WORKFORCE SEGMENTS")
    display(dept_table)

    dept_table.plot(
        kind="bar", stacked=True, figsize=(10, 5)
    )
    plt.title("Department-wise Employee Segmentation")
    plt.ylabel("Number of Employees")
    plt.tight_layout()
    plt.show()

# 4. Historical attrition analysis
if "Attrition" in data.columns:
    attrition = data["Attrition"].astype(str).str.strip().str.lower()
    data["Attrition_Flag"] = attrition.map({
        "yes": 1, "no": 0,
        "true": 1, "false": 0,
        "1": 1, "0": 0
    })

    print("\nHISTORICAL ATTRITION BY CLUSTER")
    display(
        data.groupby("KMeans_Cluster")["Attrition_Flag"]
        .agg(["count", "mean"])
        .rename(columns={
            "count": "Employees with known labels",
            "mean": "Historical attrition proportion"
        })
    )

print("\nDashboard generated successfully!")
```